

Vedant Vartak

DATA 260 - Agentic AI and Distributed Systems

Homework 5: Tools, Reliability, and a Safe Local Agent

GitHub: <https://github.com/vedantvartakwork/data260-3539>

Evidence-build commit: 52e7ec02b93249e7f3810a54608e100308da44bc

Resolved submission tag: hw5 -> 52e7ec02b93249e7f3810a54608e100308da44bc

Repository link verified. supriyaselvangesan has write access; Sbnikitha's write invitation is pending acceptance. Generated verification/report evidence follows the tagged application snapshot; application source is checked for exact tag alignment.

Item	Value	Item	Value
SID4	3539	PORT_BASE	8839
PREFIX	s3539	SEED	3539
VERIFY_SEED	263539	DOMAIN_ID	3
Domain	Grocery supply and recall notices	Local model	qwen3:8b
Hardware	Apple M4 MacBook Air	Memory / CPU	16 GB / 10 cores
Platform	macOS 15.3	Ports	API 8839 / UI 5173

This cumulative submission extends the authenticated FastAPI/MySQL application with a manufacturer relationship, Redux Toolkit and Axios, two MCP servers, strict tool schemas, bounded retries, and a local Ollama tool-calling loop with deterministic safety enforcement.

Submission contents

The repository includes RUN_LOG.txt, METRICS.md, REFLECTION.md, TOOL_CONTRACTS.md, AI_USE.md, verification.json, 150 raw fault-injection rows, MCP/agent/API raw outputs, Postman collection and environment files, and this screenshot-backed report.

1. Completion and verification

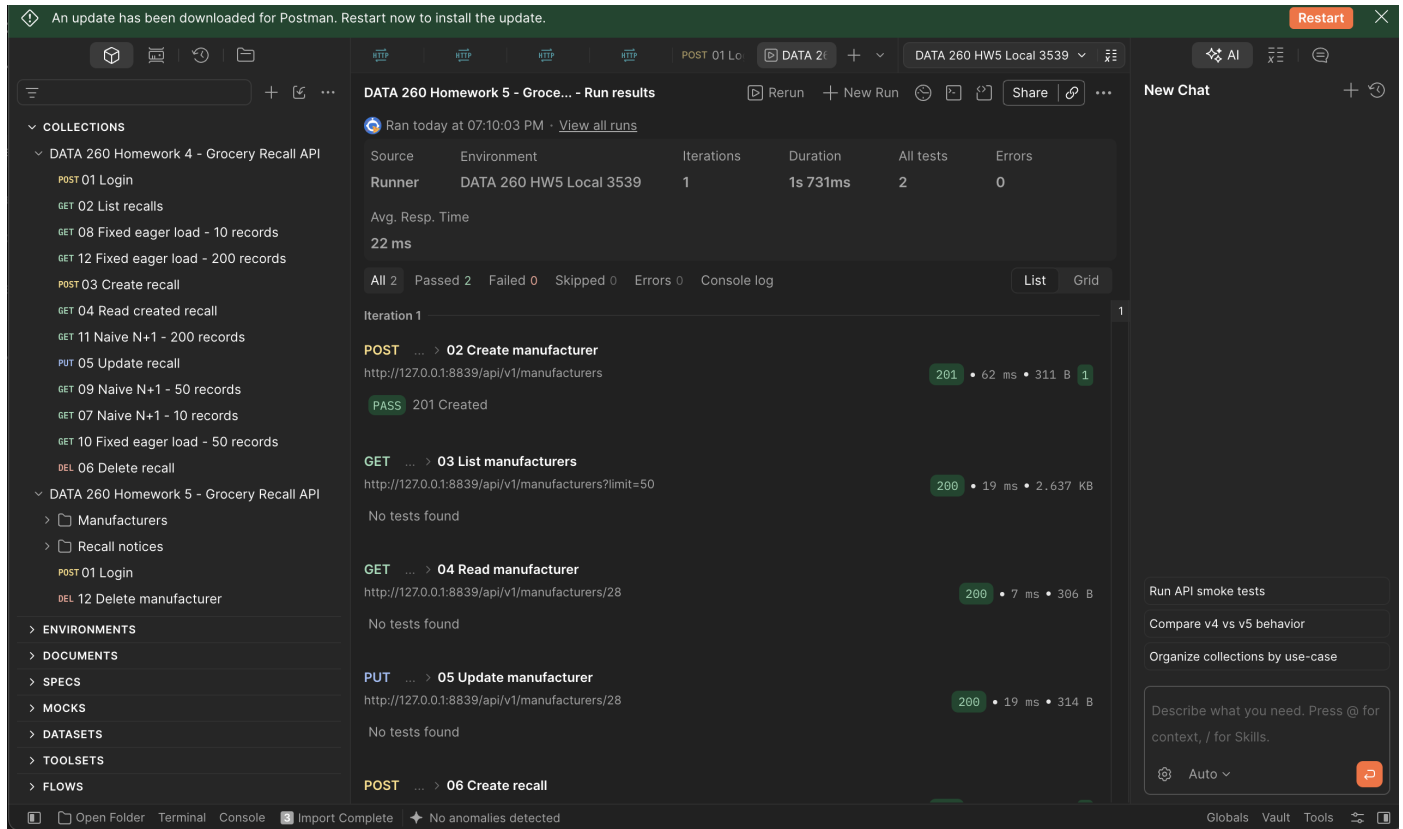
Automated local checks are summarized below; repository access is verified separately.

Area	Evidence	Result
Database/API	12 manufacturers; 500 recalls; 200 events; 14/14 authenticated checks	PASS
React/Redux	Home, create, update, delete; 10-page access to all 500 records	PASS
MealDB MCP	Exactly four live tools demonstrated	PASS
Domain MCP	Exactly search/detail/aggregate; valid and invalid calls	PASS
Reliability	150 deterministic calls; 3-attempt bounded retry	PASS
Offline tests	9/9 tool and safety tests	PASS
Cumulative suite	37/37 Python tests; React production build	PASS
Local agent	Four qwen3:8b scenarios; one enforced safety block	PASS
Verifier	14/14 checks including real STDIO and tag alignment	PASS

The verifier independently inventories both MCP servers, calls one tool from each, checks the live API health endpoint, counts all 150 experiment rows, runs the offline runner and 37-test suite, and rebuilds the React client. Both MCP calls use newly launched STDIO processes; application source is compared to the hw5 tag.

Part 1 - Database/API: actual Postman operations

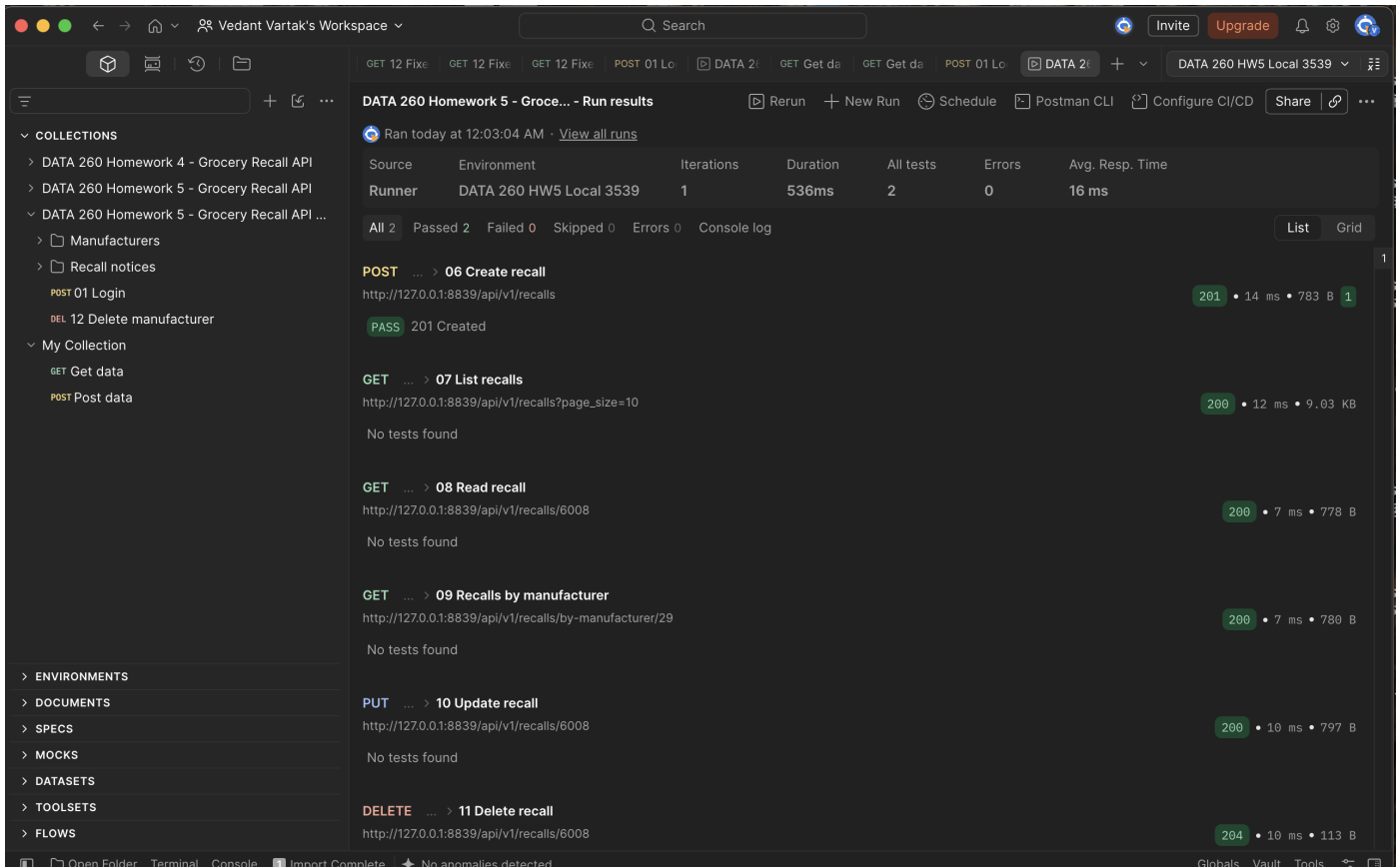
This is the original Postman application capture, not a reconstructed panel. It shows the selected HW5 environment and successful manufacturer operations. It is an earlier temporary demonstration run; the linked recall workflow in Sections 3-6 uses manufacturer 29 and recall 6008 consistently.



Postman Runner UI: HW5 collection, local environment, response statuses, timings, and test summary.

3. Actual Postman Runner evidence - recall operations

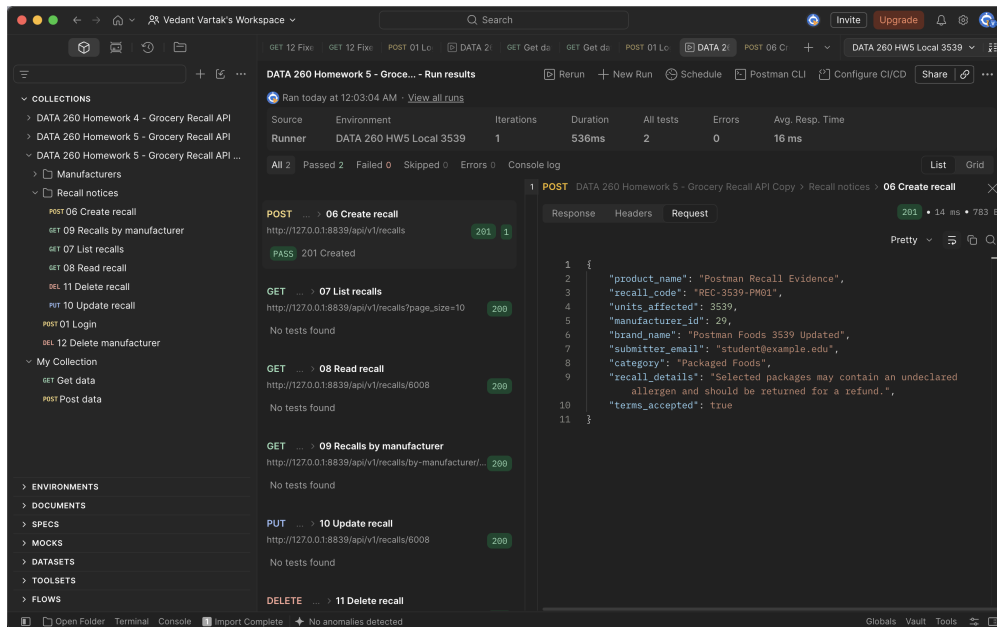
This new Postman run visibly shows recall creation, listing, reading, the manufacturer relationship endpoint, update, and delete. Login was sent before the run. It is a separate run from the manufacturer capture and companion panels: recall ID 6008 and manufacturer ID 29 identify this capture.



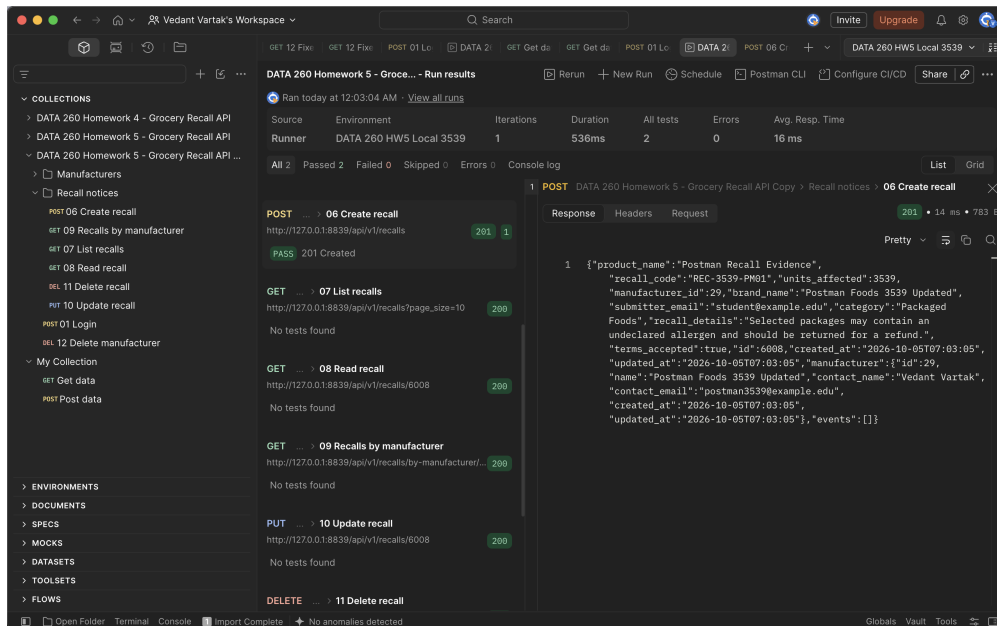
Actual Runner results: create 201; list, read, relationship, and update 200; delete 204. Two assertions passed, with zero failed assertions and zero request errors.

4. Actual Postman request and response - recall creation

These are unedited Postman application captures from the same run shown above. The first records the submitted JSON request and the second shows the stored 201 response for recall 6008.



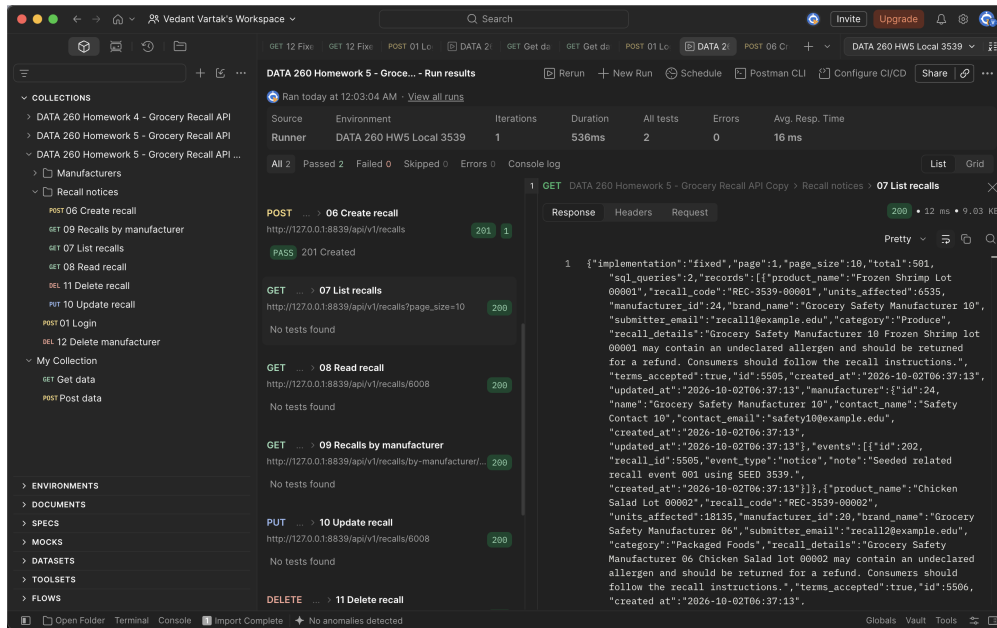
Request tab: POST /api/v1/recalls with the recall payload and 201 Created status.



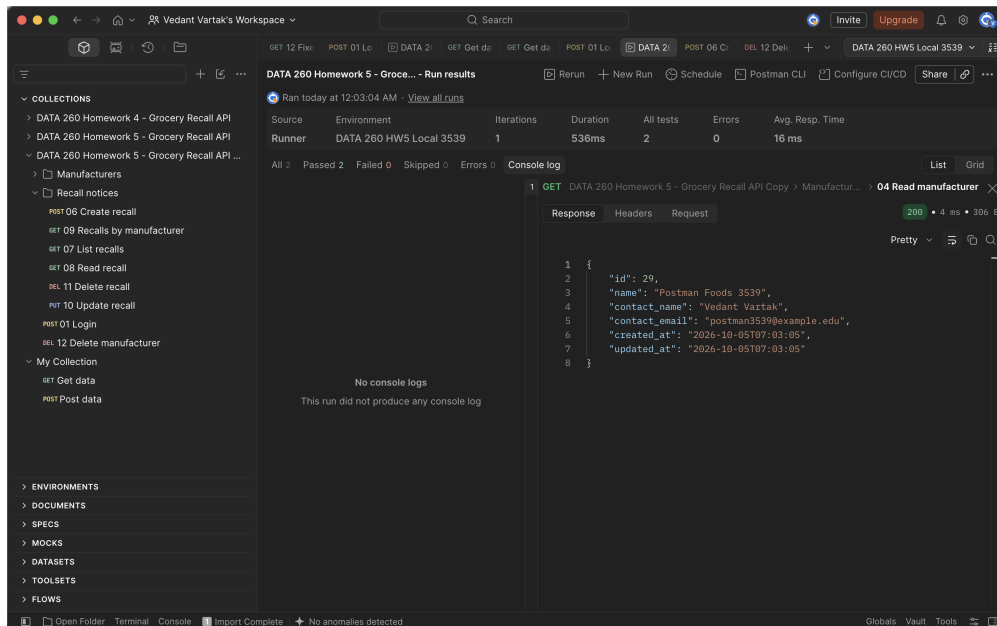
Response tab: the returned recall object, including ID 6008 and its manufacturer relation.

5. Actual Postman responses - list and direct record read

The list request demonstrates the fixed eager-load implementation and page metadata. The next capture is a completed direct-read response for the manufacturer used by the recall workflow; Section 3 independently records the matching direct recall read.



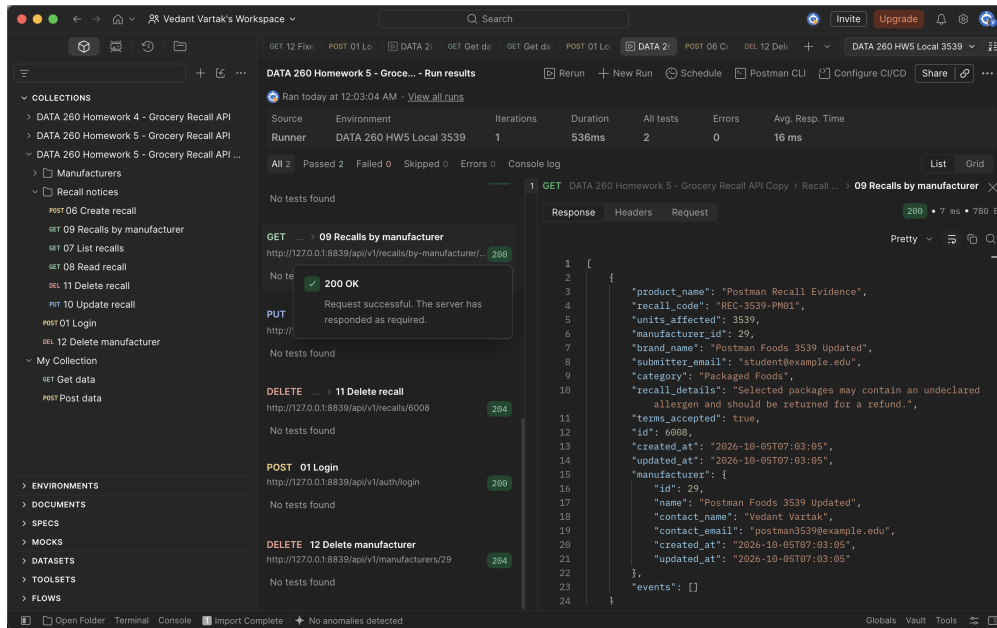
GET /api/v1/recalls?page_size=10 returned 200 with implementation metadata, total, page size, and records.



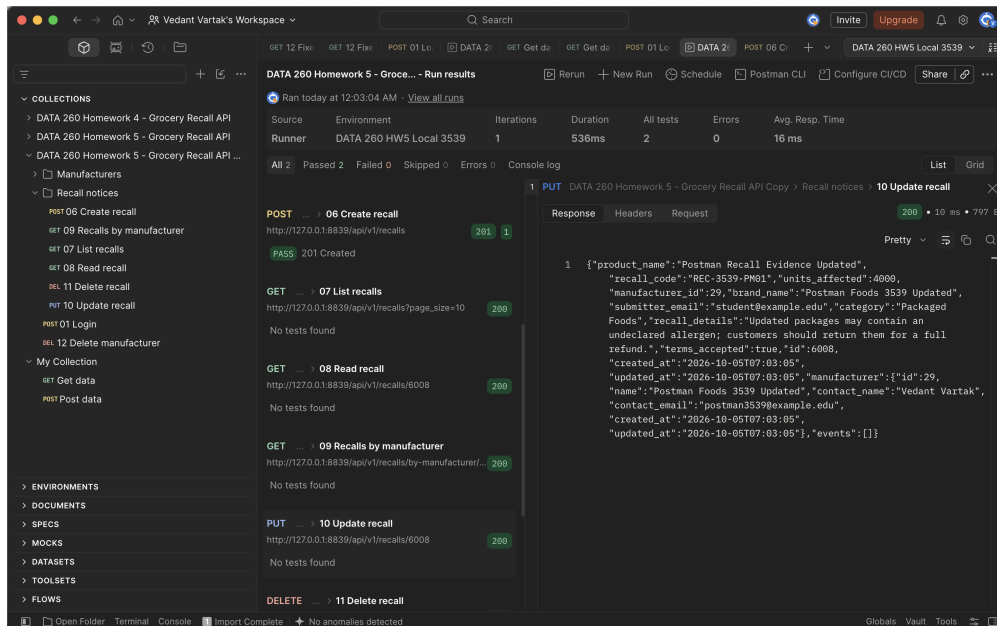
GET /api/v1/manufacturers/29 returned 200 and the complete saved manufacturer.

6. Actual Postman responses - relationship, update, and delete

The relationship query and update are captured directly in Postman. The Runner capture in Section 3 records the subsequent 204 delete response for this same recall, since a successful DELETE has no response body to display.



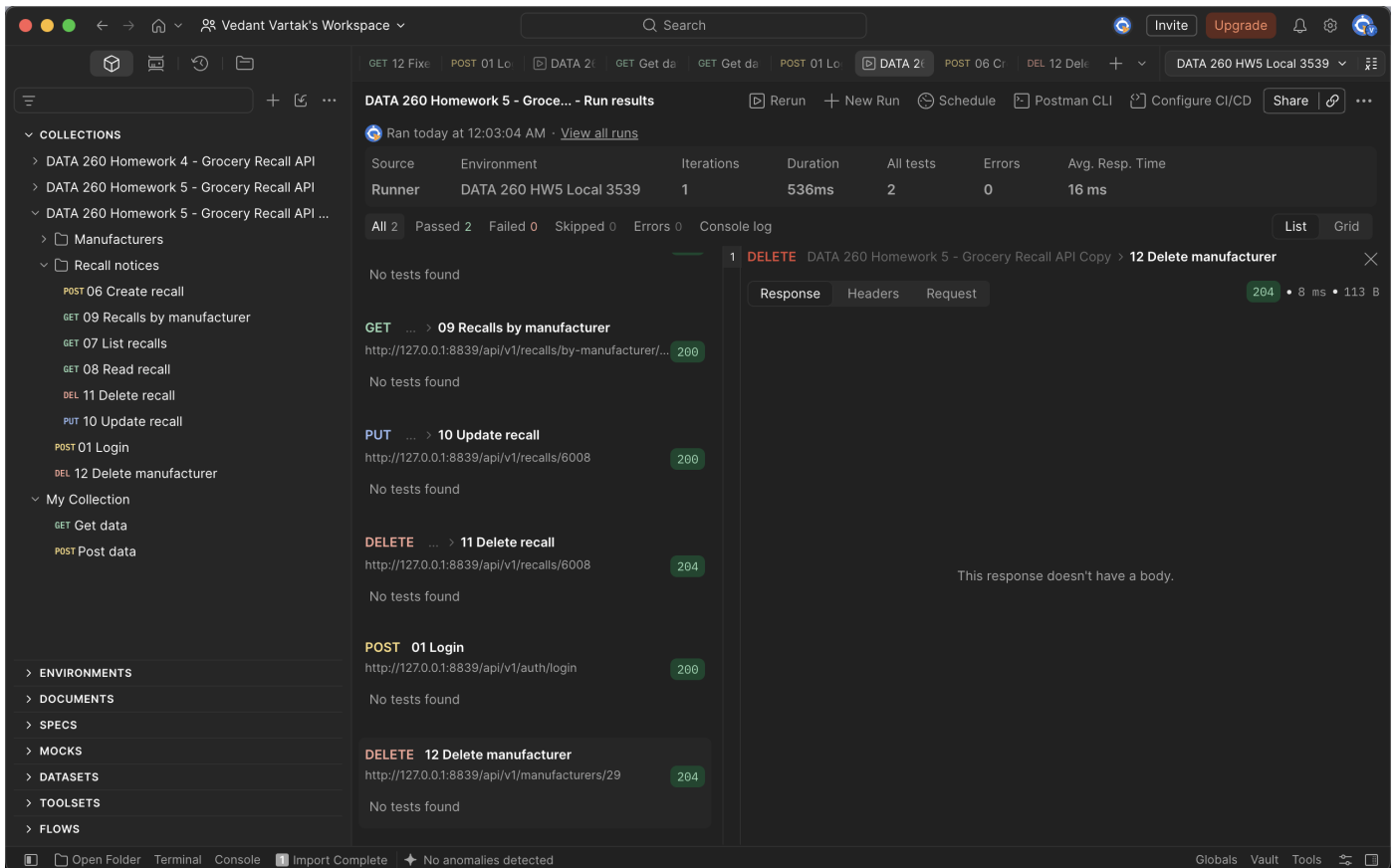
GET /api/v1/recalls/by-manufacturer/29 returned 200 and includes recall 6008 with manufacturer data.



PUT /api/v1/recalls/6008 returned 200 with the updated recall; the same run then returned DELETE 204.

6A. Actual Postman response - manufacturer deletion

This final cleanup action is shown in the real Postman Runner after its dependent recall was deleted. HTTP 204 deliberately has no response body; the completed request and status are both visible.



DELETE /api/v1/manufacturers/29 returned 204 No Content in Postman, completing the temporary demonstration data cleanup.

7. Enlarged live MySQL evidence

The database capture is given a full page so the row counts, schema, and foreign key are readable.

```
Last login: Sun Oct 4 00:34:07 on ttys018
cd /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539 && clear && .venv/bin/python scripts/capture_hw05_database_evidence.py
vedantvartak@Vedants-MacBook-Air ~ % cd /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539 && clear && .venv/bin/python scripts/capture_hw05_database_evidence.py

VEDANT VARTAK - DATA 260 HOMEWORK 5 - MYSQL EVIDENCE
timestamp_utc: 2026-10-04T07:34:11.556960+00:00
database: s3539_rel
tables: manufacturers, recall_events, recall_notices, sessions, users

row_counts:
  manufacturers: 12
  recall_notices: 500
  recall_events: 200
  users: 1
  sessions: 1

manufacturers_columns:
  id | INTEGER | nullable=False
  name | VARCHAR(160) | nullable=False
  contact_name | VARCHAR(160) | nullable=False
  contact_email | VARCHAR(255) | nullable=False
  created_at | DATETIME | nullable=False
  updated_at | DATETIME | nullable=False

recall_notices_columns:
  id | INTEGER | nullable=False
  product_name | VARCHAR(160) | nullable=False
  recall_code | VARCHAR(32) | nullable=False
  units_affected | INTEGER | nullable=False
  manufacturer_id | INTEGER | nullable=False
  brand_name | VARCHAR(160) | nullable=False
  submitter_email | VARCHAR(255) | nullable=False
  category | VARCHAR(80) | nullable=False
  recall_details | TEXT | nullable=False
  terms_accepted | TINYINT | nullable=False
  created_at | DATETIME | nullable=False
  updated_at | DATETIME | nullable=False

recall_notices_foreign_keys:
  fk_recall_notices_manufacturer: ['manufacturer_id'] -> manufacturers['id'] ondelete=RESTRICT
Saved: /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539/reports/hw05/raw/database_evidence.txt
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

12 manufacturers, 500 recalls, 200 events, required columns, and RESTRICT foreign key.

7A. Database evidence — readable detail

This close crop retains the row counts and schema columns while removing unused terminal space. The preceding page remains the complete, unmodified capture.

```
VEDANT VARTAK — DATA 260 HOMEWORK 5 — MYSQL EVIDENCE
timestamp_utc: 2026-10-04T07:34:11.556960+00:00
database: s3539_rel
tables: manufacturers, recall_events, recall_notices, sessions, users

row_counts:
  manufacturers: 12
  recall_notices: 500
  recall_events: 200
  users: 1
  sessions: 1

manufacturers_columns:
  id | INTEGER | nullable=False
  name | VARCHAR(160) | nullable=False
  contact_name | VARCHAR(160) | nullable=False
  contact_email | VARCHAR(255) | nullable=False
  created_at | DATETIME | nullable=False
  updated_at | DATETIME | nullable=False

recall_notices_columns:
  id | INTEGER | nullable=False
  product_name | VARCHAR(160) | nullable=False
  recall_code | VARCHAR(32) | nullable=False
  units_affected | INTEGER | nullable=False
  manufacturer_id | INTEGER | nullable=False
  brand_name | VARCHAR(160) | nullable=False
  submitter_email | VARCHAR(255) | nullable=False
  category | VARCHAR(80) | nullable=False
  recall_details | TEXT | nullable=False
  terms_accepted | TINYINT | nullable=False
  created_at | DATETIME | nullable=False
  updated_at | DATETIME | nullable=False

recall_notices_foreign_keys:
  fk_recall_notices_manufacturer: ['manufacturer_id'] -> manufacturers['id'] ondelete=RESTRICT
Saved: /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539/reports/hw05/raw/database_evidence.tx
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Close view of the recorded MySQL row counts and required schema columns.

8. Database and API implementation evidence

The relevant validation and transaction code is immediately followed by measured output.

DATABASE + API VALIDATION CODE AND OUTPUT

IMPLEMENTATION

RecallCreate validates positive units, email, exact category, accepted terms, unique recall_code, and manufacturer_id. The API catches IntegrityError and returns HTTP 409; the FK uses ON DELETE RESTRICT.

CODE

```
class RecallCreate(RecallInput): pass

if db.get(Manufacturer, payload.manufacturer_id) is None:
    raise HTTPException(404, 'Manufacturer not found')
try: db.commit()
except IntegrityError:
    db.rollback(); raise HTTPException(409, 'Recall code must be unique')
```

MEASURED OUTPUT

```
manufacturers=12; recall_notices=500; recall_events=200
FK: manufacturer_id -> manufacturers.id ON DELETE RESTRICT
14/14 authenticated live API checks passed
```

Validation, 409 handling, relationship integrity, and live row counts.

9. Authentication, format validation, and numeric default

The requested security and schema details are explicit rather than inferred from PASS claims.

PASSWORD, UNIQUE FORMAT, AND NUMERIC DEFAULT

PASSWORD HASHING

PBKDF2-HMAC-SHA256; 240,000 iterations; random 16-byte salt; constant-time `hmac.compare_digest` verification. Only `password_hash` is stored.

UNIQUE FIELD FORMAT

`recall_code`: `Field(pattern='^REC-[A-Z0-9]{2,23}$')`; normalized with `strip().upper()`. Database column is `unique=True` and indexed. `contact_email` and `submitter_email` use `EmailStr`.

NUMERIC DEFAULT

`units_affected`: `int = Field(default=0, ge=0, le=100_000_000)`
SQLAlchemy: `mapped_column(Integer, nullable=False, default=0)`

Salted PBKDF2 hashing, unique-field formats, and the `units_affected` default are documented from source.

10. Redux store and slice setup

The store registration, async thunk, and slice state are shown as implementation evidence.

REDUX STORE AND SLICE IMPLEMENTATION

STORE

```
export const store = configureStore({
  reducer: { recalls: recallsReducer },
});

<Provider store={store}><App /></Provider>
```

ASYNC THUNK

```
export const fetchRecalls = createAsyncThunk(
  'recalls/fetch',
  async ({ query = '', page = 1 } = {}) => {
    const response = await apiClient.get('/recalls',
      { params: { q: query || undefined, page, page_size: 50 } });
    return response.data;
  }
);
```

SLICE STATE

```
initialState: { items: [], page: 1, pageSize: 50, total: 0, status: 'idle' }
fulfilled: state.items = action.payload.records; state.total = action.payload.total
```

configureStore registers the recalls reducer; the thunk and fulfilled reducer preserve pagination metadata.

11. Redux Home - code and UI in one screenshot

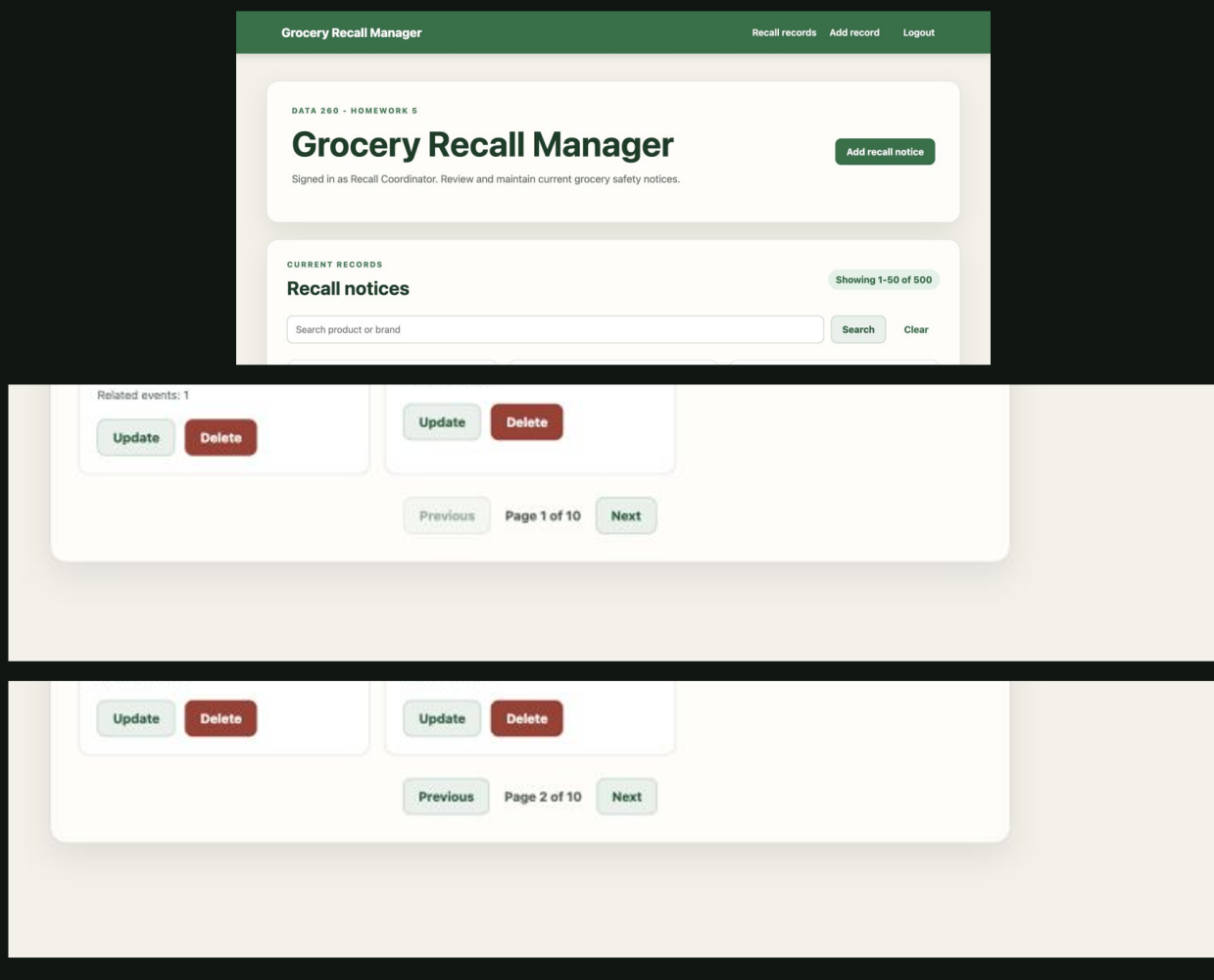
The composite contains the actual thunk/slice fields and visible UI output, including page 1 and page 2.

REDUX HOME: PAGINATED ACCESS TO ALL 500 RECORDS

RELEVANT REDUX / REACT CODE

```
export const fetchRecalls = createAsyncThunk('recalls/fetch',
  async ({ query = '', page = 1 } = {}, thunkApi) => {
    const params = { page, page_size: 50 };
    const response = await apiClient.get('/recalls', { params });
    return response.data;
  });

const { items, page, pageSize, total } = useSelector(s => s.recalls);
<span>Showing {firstShown}-{lastShown} of {total}</span>
<span>Page {page} of {totalPages}</span>
```



Showing 1-50 of 500; page 1/10 and page 2/10 prove access beyond the first 50 records.

12. Redux Create - form, code, and saved output

The create form, thunk, and persisted success result appear in one report frame.

REDUX CREATE: THUNK + SAVED UI OUTPUT

RELEVANT REDUX / REACT CODE

```
await dispatch(createRecall(payload)).unwrap();
export const createRecall = createAsyncThunk('recalls/create', async (payload) => {
  const response = await apiClient.post('/recalls', payload);
  return response.data;
});
```

The top screenshot shows a 'CREATE' form titled 'Add a recall notice'. The form includes fields for 'Product name' (HWS Evidence Granola Bars), 'Recall code (unique)' (REC-3539-90001), 'Units affected' (03539), 'Manufacturer' (Grocery Safety Manufacturer 01), 'Brand or manufacturer' (Campus Pantry Evidence), 'Submitter email' (student@example.edu), and 'Product category' (Packaged Foods). There is a 'Recall details' section with a text area containing 'Selected packages may contain an undeclared peanut allergen and should be returned for a full refund.' and a checkbox for 'I agree to the terms and conditions.'.

The bottom screenshot shows the 'CURRENT RECORDS' section titled 'Recall notices'. It features a search bar with 'HWS Evidence' entered and buttons for 'Search' and 'Clear'. A '1 shown' indicator is present. The record list shows a single entry for 'HWS Evidence Granola Bars' with recall code 'REC-3539-90001', manufacturer 'Campus Pantry Evidence', and '3,539 units affected'. It also includes the same recall details and 'Update' and 'Delete' buttons.

The completed form is visible before createRecall returns the saved record and success notice.

13. Redux Update - form, code, and revised output

The update form, thunk/reducer, and revised record output appear together.

REDUX UPDATE: THUNK + UPDATED UI OUTPUT

RELEVANT REDUX / REACT CODE

```
await dispatch(updateRecall({ id: Number(id), payload })).unwrap();
.addCase(updateRecall.fulfilled, (state, action) => {
  const index = state.items.findIndex(item => item.id === action.payload.id);
  if (index >= 0) state.items[index] = action.payload;
});
```

The image displays two screenshots of a web application interface. The top screenshot shows a form titled "Update recall #6006" with the subtitle "Edit the record and save it back to MySQL." The form contains several input fields: "Product name" (HWS Evidence Granola Bars Updated), "Recall code (unique)" (REC-3539-90001), "Units affected" (04540), "Manufacturer" (Grocery Safety Manufacturer 01), "Brand or manufacturer" (Campus Pantry Evidence Updated), "Submitter email" (student@example.edu), and "Product category" (Packaged Foods). Below these fields is a "Recall details" section with a text area containing the message: "Updated packages may contain an undeclared peanut allergen. Customers should return them for a full refund." At the bottom of the form is a checkbox labeled "I agree to the terms and conditions." The bottom screenshot shows the result of the update. A green success message at the top reads "Updated recall REC-3539-90001 successfully." Below this is a section titled "CURRENT RECORDS" with a sub-header "Recall notices". It features a search bar with the placeholder "Search product or brand" and buttons for "Search" and "Clear". A single record is displayed, showing the same details as the form above, including the product name, recall code, units affected, manufacturer, brand, submitter email, product category, and recall details. At the bottom of the record are "Update" and "Delete" buttons.

The edited form is visible before Redux replaces the item with the API response.

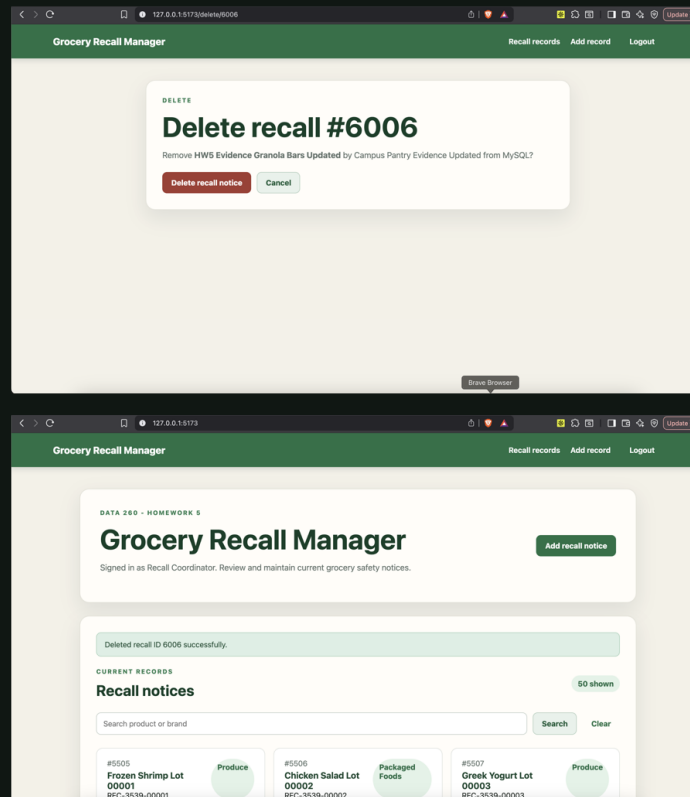
14. Redux Delete - code, confirmation, and output

The dedicated confirmation route and post-delete state appear with the reducer code in one composite screenshot.

REDUX DELETE: CONFIRMATION ROUTE + UI OUTPUT

RELEVANT REDUX / REACT CODE

```
<Link to={`/delete/${record.id}`}>Delete</Link>
await dispatch(deleteRecall(Number(id))).unwrap();
.addCase(deleteRecall.fulfilled, (state, action) => {
  state.items = state.items.filter(item => item.id !== action.payload);
  state.notice = `Deleted recall ID ${action.payload} successfully.`;
});
```



Explicit confirmation precedes deletion; the success notice follows API completion.

Part 2A - MealDB MCP implementation

This source evidence shows the exact four-tool surface, stdio transport, stderr logging, HTTP timeout, and handled failures.

MEALDB MCP SERVER IMPLEMENTATION

STDIO + STDERR

```
logging.basicConfig(stream=sys.stderr, level=logging.INFO)
mcp = FastMCP('meals')
if __name__ == '__main__': mcp.run(transport='stdio')
```

TIMEOUT + ERROR HANDLING

```
try:
    response = httpx.get(url, params=params, timeout=10.0)
    response.raise_for_status()
    payload = response.json()
except (httpx.HTTPError, ValueError) as exc:
    LOGGER.error('TheMealDB request failed: %s', exc)
    raise RuntimeError(f'TheMealDB request failed: {exc}') from exc
```

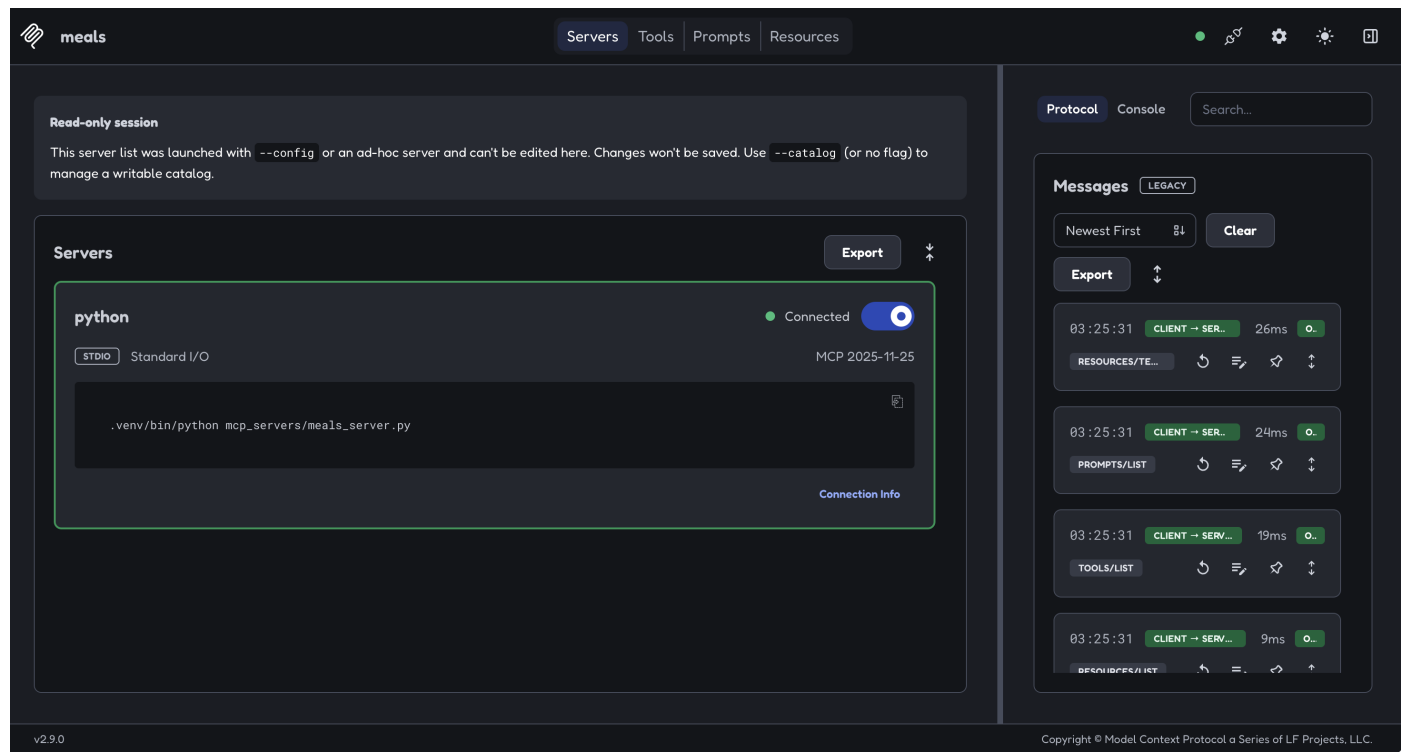
EXACT TOOL SURFACE

```
@mcp.tool search_meals_by_name
@mcp.tool meals_by_ingredient
@mcp.tool meal_details
@mcp.tool random_meal
```

MealDB server implementation excerpts and failure boundary.

16. MealDB MCP server connection

The Inspector connected over stdio using the project virtual environment and exposed exactly four required tools.



Connected MealDB MCP server in Inspector; tools, prompts, and resources inventories completed successfully.

17. MealDB Inspector — search_meals_by_name

The actual Inspector view visibly includes the submitted arguments and returned output.

The screenshot displays the MealDB Inspector interface. On the left, the 'Tools' panel lists available tools, with 'search_meals_by_name' selected. The 'Results' panel shows the raw JSON response for the tool call. The 'Messages' panel on the right is expanded, showing the 'Protocol' tab with the full JSON-RPC message, including arguments, meta, and structured content.

Tools

- Search tools...
- Paginated
- search_meals_by_name**
- meals_by_ingredient
- meal_details
- random_meal

Results

```
1 {
2   "id": "52771",
3   "name": "Spicy Arrabiata Penne",
4   "area": "Italian",
5   "category": "Vegetarian",
6   "thumb": "https://www.themeadb.com/images/52771.jpg"
7 }
```

Structured Output

```
1 {
2   "result": [
3     {
4       "id": "52771",
5       "name": "Spicy Arrabiata Penne",
6       "area": "Italian",
7       "category": "Vegetarian",
8       "thumb": "https://www.themeadb.com/images/52771.jpg"
9     }
10  ]
11 }
```

Messages (LEGACY)

Newest First | Clear

Export

Copy

```
{
  "arguments": {
    "limit": 5,
    "query": "Arrabiata"
  },
  "_meta": {
    "progressToken": 5
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "{\n  \"id\": \"52771\", \n}"
      }
    ],
    "structuredContent": {
      "result": [
        {
          "id": "52771",
          "name": "Spicy Arrabiata Penne",
          "area": "Italian",
          "category": "Vegetarian",
          "thumb": "https://www.themeadb.com/images/52771.jpg"
        }
      ]
    }
  }
}
```

v2.9.0 Copyright © Model Context Protocol a Series of LF Projects, LLC.

Actual MCP Inspector call for `search_meals_by_name`, with the Protocol pane expanded to show its exact arguments and response.

17A. MealDB result — search_meals_by_name

The exact companion panel is placed on its own page so the result is readable at normal viewing size.

MEALDB MCP TOOL 1/4: search_meals_by_name

INPUT JSON

```
{
  "query": "Arrabiata",
  "limit": 5
}
```

STRUCTURED OUTPUT

```
{
  "result": [
    {
      "id": "52771",
      "name": "Spicy Arrabiata Penne",
      "area": "Italian",
      "category": "Vegetarian",
      "thumb": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg"
    }
  ]
}
```

Exact input JSON and readable output for search_meals_by_name; complete raw output is retained.

18. MealDB Inspector — meals_by_ingredient

The actual Inspector view visibly includes the submitted arguments and returned output.

The screenshot displays the MealDB Inspector interface. On the left, the 'Tools' panel lists available tools: 'search_meals_by_name', 'meals_by_ingredient' (selected), 'meal_details', and 'random_meal'. The 'Results' panel shows three JSON objects representing meal entries. The 'Structured Output' panel displays the full JSON response from the 'meals_by_ingredient' tool. The 'Protocol' pane on the right is expanded, showing the 'CLIENT → SERVER' message with the arguments and the response.

```
1 {
2   "id": "52940",
3   "name": "Brown Stew Chicken",
4   "thumb": "https://www.themeadb.com/images/52940.jpg"
5 }
```

```
1 {
2   "id": "53161",
3   "name": "Chicken & chorizo rice pot",
4   "thumb": "https://www.themeadb.com/images/53161.jpg"
5 }
```

```
1 {
2   "id": "52846",
3   "name": "Chicken & mushroom Hotpot",
4   "thumb": "https://www.themeadb.com/images/52846.jpg"
5 }
```

```
1 {
2   "result": [
3     {
4       "id": "52940",
5       "name": "Brown Stew Chicken",
6       "thumb": "https://www.themeadb.com/images/52940.jpg"
7     },
8     {
9       "id": "53161",
10      "name": "Chicken & chorizo rice pot",
11      "thumb": "https://www.themeadb.com/images/53161.jpg"
12    }
13   ]
14 }
```

```
{
  "name": "meals_by_ingredient",
  "arguments": {
    "limit": 3,
    "ingredient": "chicken"
  },
  "_meta": {
    "progressToken": 6
  }
}
```

Version: v2.9.0 | Copyright © Model Context Protocol a Series of LF Projects, LLC.

Actual MCP Inspector call for meals_by_ingredient, with the Protocol pane expanded to show its exact arguments and response.

18A. MealDB result — meals_by_ingredient

The exact companion panel is placed on its own page so the result is readable at normal viewing size.

MEALDB MCP TOOL 2/4: meals_by_ingredient

INPUT JSON

```
{
  "ingredient": "chicken",
  "limit": 3
}
```

STRUCTURED OUTPUT

```
{
  "result": [
    {
      "id": "52940",
      "name": "Brown Stew Chicken",
      "thumb": "https://www.themealdb.com/images/media/meals/sypxpx1515365095.jpg"
    },
    {
      "id": "53161",
      "name": "Chicken & chorizo rice pot",
      "thumb": "https://www.themealdb.com/images/media/meals/fk80jp1763280767.jpg"
    },
    {
      "id": "52846",
      "name": "Chicken & mushroom Hotpot",
      "thumb": "https://www.themealdb.com/images/media/meals/uuuspp1511297945.jpg"
    }
  ]
}
```

Exact input JSON and readable output for meals_by_ingredient; complete raw output is retained.

19. MealDB Inspector — meal_details

The actual Inspector view visibly includes the submitted arguments and returned output.

The screenshot displays the MealDB Inspector interface. On the left, the 'Tools' panel lists several tools, with 'meal_details' selected. The 'Results' panel in the center shows the JSON response for the 'meal_details' tool call. On the right, the 'Protocol' pane is expanded, showing the client request and the server response for the 'meal_details' tool call.

Tools

- Search tools...
- Paginated
- search_meals_by_name
- meals_by_ingredient
- meal_details**
- random_meal

Results

```
1 {
2   "id": "52771",
3   "name": "Spicy Arrabiata Penne",
4   "category": "Vegetarian",
5   "area": "Italian",
6   "instructions": "Bring a large pot of wat",
7   "image": "https://www.themealdb.com/image",
8   "source": null,
9   "youtube": "https://www.youtube.com/watch",
10  "ingredients": [
11    {
12      "name": "penne rigate",
13      "measure": "1 pound"
14    },
15    {
16      "name": "olive oil",
17      "measure": "1/4 cup"
18    },
19    {
20      "name": "garlic",
21      "measure": "3 cloves"
22    },
23    {
24      "name": "chopped tomatoes",
25      "measure": "1 tin"
26    },
27    {
28      "name": "red chilli flakes",
29      "measure": "1/2 teaspoon"
30    },
31    {
32      "name": "italian seasoning",
33      "measure": "1/2 teaspoon"
34    }
35  ]
36 }
```

Protocol Console Search...

Messages LEGACY

Newest First Clear

Export

CLIENT → SERVER 912ms

meal_details

name: "meal_details",
arguments: {
 "id": "52771"
},
_meta: {
 "progressToken": 7
}

"jsonrpc": "2.0",
"id": 7,
"result": {

Actual MCP Inspector call for meal_details, with the Protocol pane expanded to show its exact arguments and response.

19A. MealDB result — meal_details

The exact companion panel is placed on its own page so the result is readable at normal viewing size.

MEALDB MCP TOOL 3/4: meal_details

INPUT JSON

```
{
  "id": "52771"
}
```

STRUCTURED OUTPUT

```
{
  "id": "52771",
  "name": "Spicy Arrabiata Penne",
  "category": "Vegetarian",
  "area": "Italian",
  "instructions": "Bring a large pot of water to a boil. Add kosher salt to the boiling water, then add the pasta. Cook according to the package instructions, about 9 minutes.\r\n\r\nIn a large skillet over medium-high heat, add the olive oil and heat until the oil starts to shimmer. Add the garlic and cook, stirring, until fragrant, 1 to 2 minutes. Add the chopped tomatoes, red chile flakes, Italian seasoning and salt and pepper to taste. Bring to a boil and cook for 5 minutes. Remove from the heat and add the chopped basil.\r\n\r\nDrain the pasta and add it to the sauce. Garnish with Parmigiano-Reggiano flakes and more basil and serve warm.",
  "image": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg",
  "source": null,
  "youtube": "https://www.youtube.com/watch?v=1lszT_gul08",
  "ingredients": [
    {"name": "penne rigate", "measure": "1 pound"},
    {"name": "olive oil", "measure": "1/4 cup"},
    {"name": "garlic", "measure": "3 cloves"},
    {"name": "chopped tomatoes", "measure": "1 tin"},
    {"name": "red chilli flakes", "measure": "1/2 teaspoon"},
    {"name": "italian seasoning", "measure": "1/2 teaspoon"},
    {"name": "basil", "measure": "6 leaves"},
    {"name": "Parmigiano-Reggiano", "measure": "sprinkling"}
  ]
}
```

Exact input JSON and readable output for meal_details; complete raw output is retained.

20. MealDB Inspector — random_meal

The actual Inspector view visibly includes the submitted arguments and returned output.

The screenshot displays the MealDB Inspector interface. On the left, the 'Tools' panel lists various tools, with 'random_meal' selected. The 'Results' panel in the center shows the JSON response for the 'random_meal' tool call. The 'Protocol' panel on the right is expanded, showing the client request and the server response.

Tools

- Search tools...
- Paginated
- search_meals_by_name
- meals_by_ingredient
- meal_details
- random_meal**

Results

```
1 {
2   "id": "53035",
3   "name": "Ham hock colcannon",
4   "category": "Pork",
5   "area": "Irish",
6   "instructions": "STEP 1\r\nPeel and cut t
7   "image": "https://www.themealdb.com/image
8   "source": "https://www.bbcgoodfood.com/re
9   "youtube": "https://www.youtube.com/watch
10  "ingredients": [
11    {
12      "name": "Floury Potatoes",
13      "measure": "800g"
14    },
15    {
16      "name": "Butter",
17      "measure": "50g"
18    },
19    {
20      "name": "Garlic Clove",
21      "measure": "3 chopped"
22    },
23    {
24      "name": "Cabbage",
25      "measure": "1 chopped"
26    },
27    {
28      "name": "Spring Onions",
29      "measure": "8"
30    },
31    {
32      "name": "Double Cream",
33      "measure": "100ml"
34    }
35  ]
36 }
```

Protocol Console Search...

Messages LEGACY

Newest First Clear

Export

CLIENT → SERVER 1138ms

random_meal

S:

```
{
  "name": "random_meal",
  "arguments": {},
  "_meta": {
    "progressToken": 8
  }
}
```

"jsonrpc": "2.0",
"id": 8,
"result": {
 "content": [
 ,

Actual MCP Inspector call for random_meal, with the Protocol pane expanded to show its exact arguments and response.

20A. MealDB result — random_meal

The exact companion panel is placed on its own page so the result is readable at normal viewing size.

MEALDB MCP TOOL 4/4: random_meal

INPUT JSON

```
{}
```

STRUCTURED OUTPUT

```
{
  "id": "53035",
  "name": "Ham hock colcannon",
  "category": "Pork",
  "area": "Irish",
  "instructions": "STEP 1\nPeel and cut the potatoes into even, medium-sized chunks. Put in a large pan filled with cold salted water, bring to the boil and cook for 10-15 mins until a knife can be inserted into the potatoes easily.\n\nSTEP 2\nMeanwhile, melt the butter in a large sauté pan over a medium heat. Add the garlic, cabbage, spring onions and some seasoning. Stir occasionally until the cabbage is wilted but still retains a little bite, then set aside.\n\nSTEP 3\nDrain the potatoes, leave to steam-dry for a couple of mins, then mash with the cream, mustard and seasoning in the same saucepan. Stir in the cabbage and ham hock. Keep warm over a low heat.\n\nSTEP 4\nReheat the pan you used to cook the cabbage (no need to wash first), add a splash of oil, crack in the eggs and fry to your liking. To serve, divide the colcannon between bowls and top each with a fried egg.",
  "image": "https://www.themealdb.com/images/media/meals/n41ny81608588066.jpg",
  "source": "https://www.bbcgoodfood.com/recipes/ham-hock-colcannon",
  "youtube": "https://www.youtube.com/watch?v=aQJAU2iJ730",
  "ingredients": [
    {"name": "Floury Potatoes", "measure": "800g"},
    {"name": "Butter", "measure": "50g"},
    {"name": "Garlic Clove", "measure": "3 chopped"},
    {"name": "Cabbage", "measure": "1 chopped"},
    {"name": "Spring Onions", "measure": "8"},
    {"name": "Double Cream", "measure": "100ml"},
    {"name": "Mustard", "measure": "2 tbs"},
    {"name": "Ham", "measure": "180g"},
    {"name": "Eggs", "measure": "4"}
  ]
}
```

Exact input JSON and readable output for random_meal; complete raw output is retained.

Part 2B - Domain MCP implementation

The server exposes exactly three stdio tools and routes all calls through the validated shared boundary.

DOMAIN MCP SERVER IMPLEMENTATION

STDIO + STDERR

```
logging.basicConfig(stream=sys.stderr, level=logging.INFO)
mcp = FastMCP('s3539_rel-recall-tools')
if __name__ == '__main__': mcp.run(transport='stdio')
```

EXACT THREE TOOLS

```
@mcp.tool(name='search')
def search(query: str, limit: int = 5): ...

@mcp.tool(name='detail')
def detail(recall_id: int): ...

@mcp.tool(name='aggregate')
def aggregate(group_by: str, min_units: int = 0): ...
```

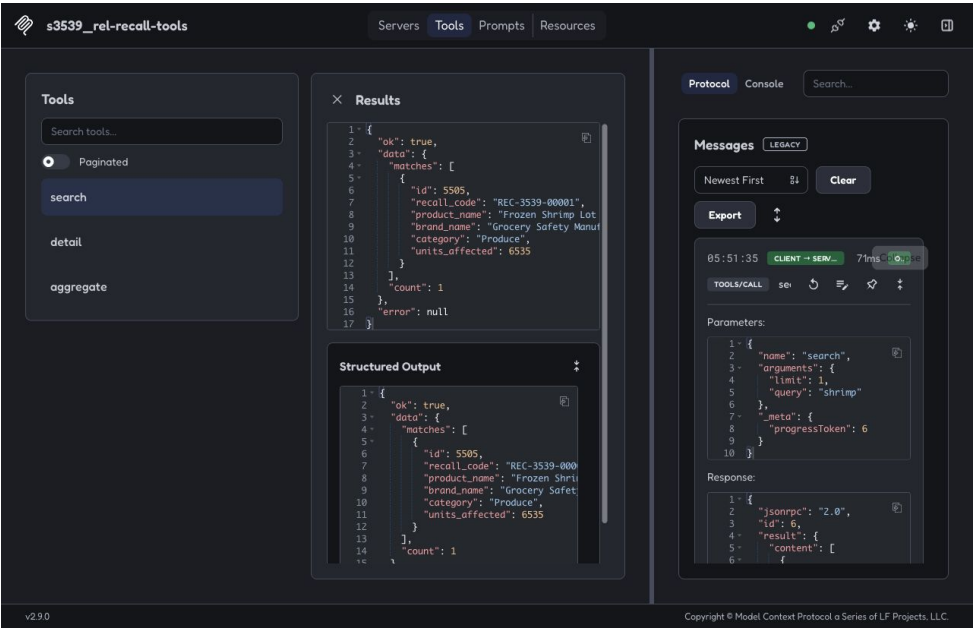
BOUNDARY ERROR ENVELOPE

```
call_domain_tool validates typed inputs and catches ValidationError, returning the same JSON-serializable
{ok, data, error} shape for success and failure.
```

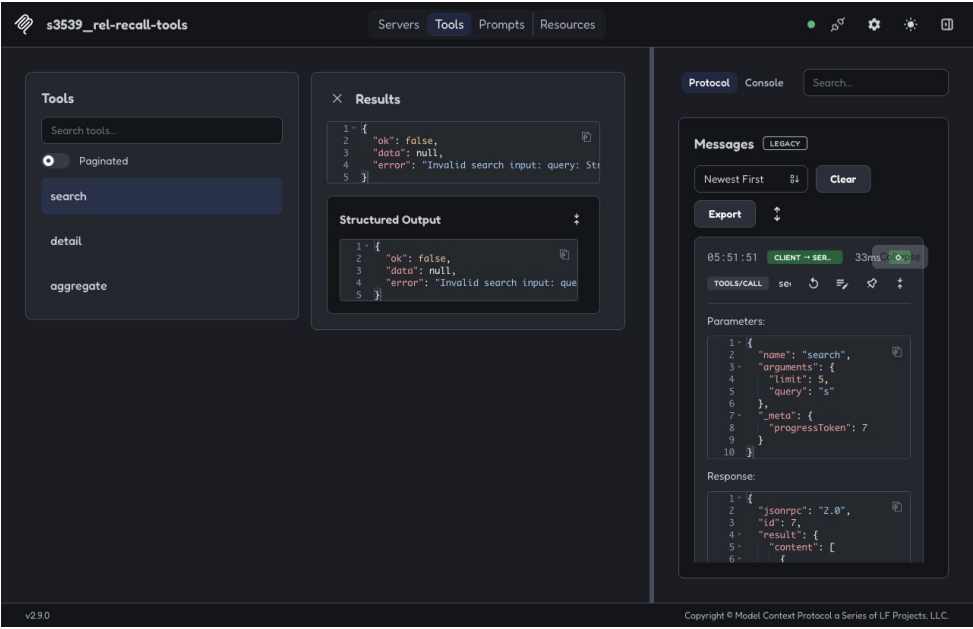
stderr logging, stdio transport, exact tool signatures, and stable error envelope.

22. Actual Domain Inspector calls - search

The expanded Protocol panes show exact inputs and responses for one valid and one rejected call.



Actual valid search call in MCP Inspector, including submitted arguments and response.



Actual invalid search call, rejected arguments, and complete failure envelope in MCP Inspector.

23. Domain MCP contract - search

Expected JSON schema, valid input/output, rejected JSON, complete error, and rejection reason are shown together.

DOMAIN MCP CONTRACT 1/3: search

EXPECTED JSON INPUT SCHEMA

```
{("type":"object","additionalProperties":false,"required":["query"],"properties":{"query":{"type":"string","minLength":2,"maxLength":80},"limit":{"type":"integer","minimum":1,"maximum":25,"default":5}}}
```

VALID INPUT

```
{  "query": "shrimp",  "limit": 1}
```

VALID OUTPUT

```
{  "ok": true,  "data": {    "matches": [      {        "id": 5505,        "recall_code": "REC-3539-00001",        "product_name": "Frozen Shrimp Lot 00001",        "brand_name": "Grocery Safety Manufacturer 10",        "category": "Produce",        "units_affected": 6535      }    ],    "count": 1  },  "error": null}
```

REJECTED JSON OBJECT

```
{  "query": "s",  "limit": 5}
```

COMPLETE RETURNED ERROR

```
{  "ok": false,  "data": null,  "error": "Invalid search input: query: String should have at least 2 characters"}
```

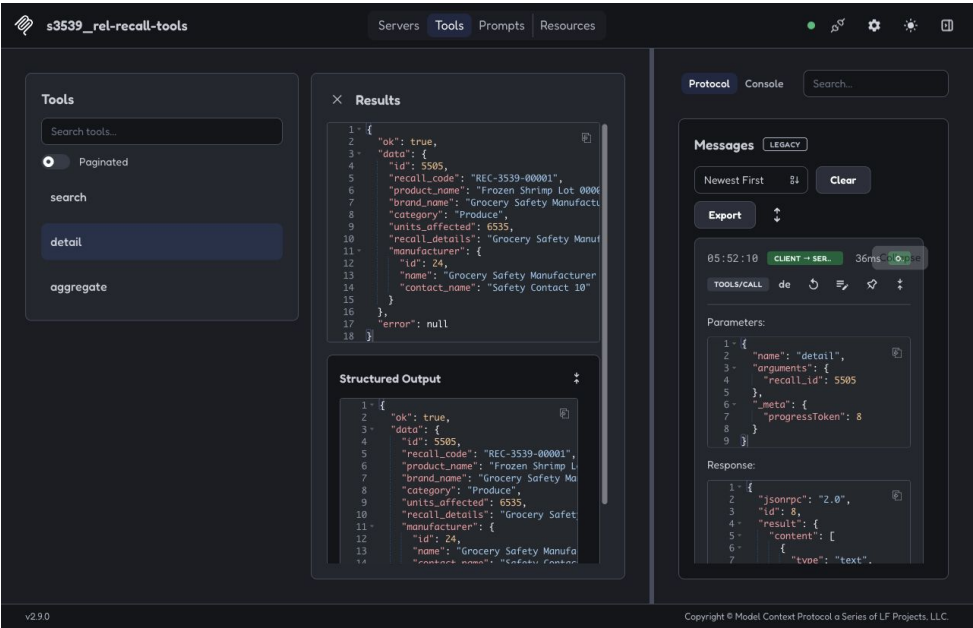
WHY REJECTED

One character is below the declared minimum and creates an excessively broad lookup.

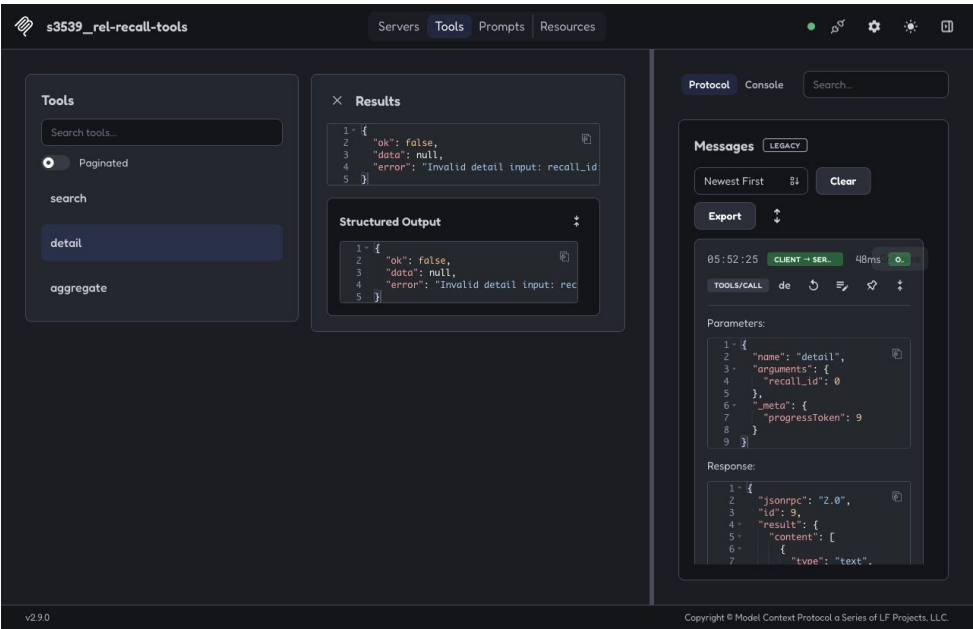
The search contract returns the stable {ok, data, error} envelope for both valid and invalid calls.

24. Actual Domain Inspector calls - detail

The expanded Protocol panes show exact inputs and responses for one valid and one rejected call.



Actual valid detail call in MCP Inspector, including submitted arguments and response.



Actual invalid detail call, rejected arguments, and complete failure envelope in MCP Inspector.

25. Domain MCP contract - detail

Expected JSON schema, valid input/output, rejected JSON, complete error, and rejection reason are shown together.

DOMAIN MCP CONTRACT 2/3: detail

EXPECTED JSON INPUT SCHEMA

```
{("type":"object","additionalProperties":false,"required":["recall_id"],"properties":{"recall_id":{"type":"integer","exclusiveMinimum":0}}}
```

VALID INPUT

```
{
  "recall_id": 5505
}
```

VALID OUTPUT

```
{
  "ok": true,
  "data": {
    "id": 5505,
    "recall_code": "REC-3539-00001",
    "product_name": "Frozen Shrimp Lot 00001",
    "brand_name": "Grocery Safety Manufacturer 10",
    "category": "Produce",
    "units_affected": 6535,
    "recall_details": "Grocery Safety Manufacturer 10 Frozen Shrimp lot 00001 may contain an undeclared allergen and should be returned for a refund. Consumers should follow the recall instructions.",
    "manufacturer": {
      "id": 24,
      "name": "Grocery Safety Manufacturer 10",
      "contact_name": "Safety Contact 10"
    }
  },
  "error": null
}
```

REJECTED JSON OBJECT

```
{
  "recall_id": 0
}
```

COMPLETE RETURNED ERROR

```
{
  "ok": false,
  "data": null,
  "error": "Invalid detail input: recall_id: Input should be greater than 0"
}
```

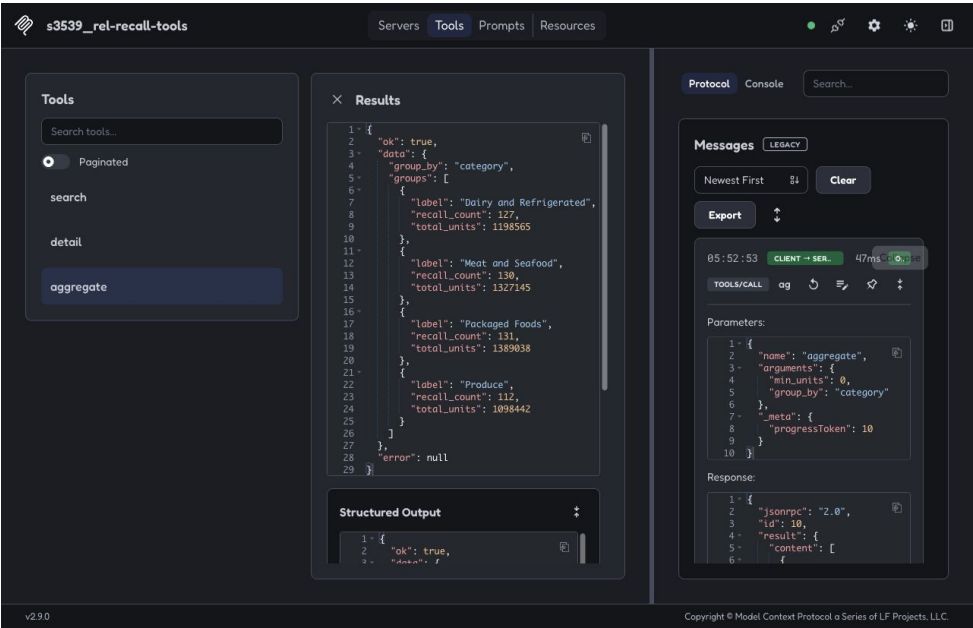
WHY REJECTED

Database IDs begin at 1, so zero is outside the accepted domain.

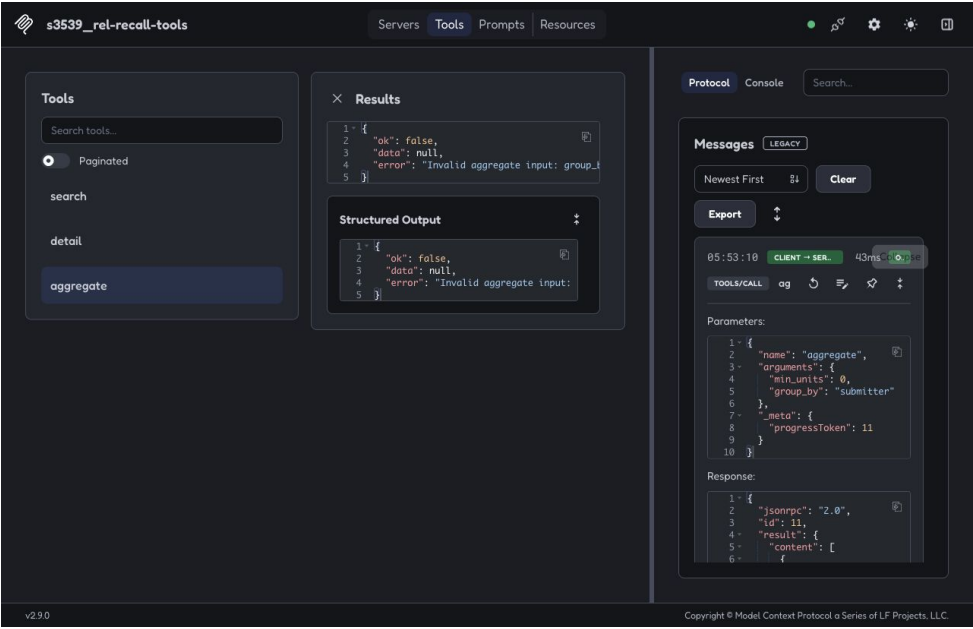
The detail contract returns the stable {ok, data, error} envelope for both valid and invalid calls.

26. Actual Domain Inspector calls - aggregate

The expanded Protocol panes show exact inputs and responses for one valid and one rejected call.



Actual valid aggregate call in MCP Inspector, including submitted arguments and response.



Actual invalid aggregate call, rejected arguments, and complete failure envelope in MCP Inspector.

27. Domain MCP contract - aggregate

Expected JSON schema, valid input/output, rejected JSON, complete error, and rejection reason are shown together.

DOMAIN MCP CONTRACT 3/3: aggregate

EXPECTED JSON INPUT SCHEMA

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["group_by"],
  "properties": {
    "group_by": {
      "type": "string",
      "pattern": "^(category|manufacturer)$",
      "min_units": {
        "type": "integer",
        "minimum": 0,
        "maximum": 1000000,
        "default": 0
      }
    }
  }
}
```

VALID INPUT

```
{
  "group_by": "category",
  "min_units": 0
}
```

VALID OUTPUT

```
{
  "ok": true,
  "data": {
    "group_by": "category",
    "groups": [
      {
        "label": "Dairy and Refrigerated",
        "recall_count": 127,
        "total_units": 1198565
      },
      {
        "label": "Meat and Seafood",
        "recall_count": 130,
        "total_units": 1327145
      },
      {
        "label": "Packaged Foods",
        "recall_count": 131,
        "total_units": 1389038
      },
      {
        "label": "Produce",
        "recall_count": 112,
        "total_units": 1098442
      }
    ]
  },
  "error": null
}
```

REJECTED JSON OBJECT

```
{
  "group_by": "submitter",
  "min_units": 0
}
```

COMPLETE RETURNED ERROR

```
{
  "ok": false,
  "data": null,
  "error": "Invalid aggregate input: group_by: String should match pattern \"^(category|manufacturer)$\""
}
```

WHY REJECTED

submitter is not an approved grouping dimension and could expose personal contact data.

The aggregate contract returns the stable {ok, data, error} envelope for both valid and invalid calls.

Part 3 - Operation timeout, retries, and stress

The operation is executed through `Future.result(timeout=remaining)`, so a hanging call is interrupted from the caller's perspective rather than merely checked after failure.

RETRY IMPLEMENTATION AND MEASURED OUTPUT

ACTUAL OPERATION TIMEOUT

```
remaining = policy.timeout_seconds - (clock() - started)
executor = ThreadPoolExecutor(max_workers=1)
future = executor.submit(operation)
try:
    data = future.result(timeout=remaining)
except FutureTimeoutError:
    future.cancel()
    executor.shutdown(wait=False, cancel_futures=True)
    last_error = TimeoutError('operation timed out')
    break
```

POLICY

```
max_attempts=3; timeout=100ms; delays=1ms, 2ms; cap=4ms; VERIFY_SEED=263539
```

RUNTIME ASSERTION

A deliberately sleeping 200ms operation is called with a 30ms deadline. The test asserts `ok=false`, error contains 'timed out', and caller elapsed time is under 120ms.

FAULT OUTPUT

```
150 total calls (50 each at 0%, 20%, 50%)
Observed: immediate success; failure|success; failure|failure|failure
```

Runtime test: a 200 ms operation returns a clean timeout within the 30 ms caller deadline; retries remain bounded.

29. Fault-injection metrics and representative rows

The experiment used VERIFY_SEED 263539 and exactly 150 calls: 50 each at 0%, 20%, and 50% injected failure.

Injected failure	Calls	Success	Mean latency	p99 latency
0%	50	100%	0.031 ms	0.071 ms
20%	50	98%	0.323 ms	3.982 ms
50%	50	92%	0.984 ms	4.026 ms

Policy: at most three attempts, 100 ms total timeout, and bounded exponential delays of 1 ms then 2 ms (4 ms cap). p99 uses nearest rank (the maximum of 50 samples). Normal calls have negligible delay. At 50% injected failure, 92% still completed, while the remaining failures returned a clean error after exhausting the fixed attempt budget. A batch workload could tolerate more attempts and jitter; the interactive assistant favors a short, predictable response ceiling.

The numbers in this table are from the final 150-call run in raw/fault_injection_calls.csv and its matching raw/fault_injection_summary.json. Representative rows in the CSV include immediate success, failure-then-success, and three exhausted attempts.

Three measured retry outcomes from the same run

First-attempt success

```
seed=263539; failure_rate=0.0; call=1
success=True; attempts=1; latency_ms=0.071
outcomes=success
error=null
```

Failure followed by retry success

```
seed=263539; failure_rate=0.2; call=13
success=True; attempts=2; latency_ms=1.4
outcomes=failure|success
error=null
```

All allowed attempts exhausted

```
seed=263539; failure_rate=0.2; call=46
success=False; attempts=3; latency_ms=3.941
outcomes=failure|failure|failure
error=operation failed after 3 attempts: injected transient storage failure
```

Part 4 - execute_tool and offline assertions

Implementation and representative assertions are immediately followed by the full named PASS output.

SAFE execute_tool + OFFLINE ASSERTIONS

execute_tool CODE

```
if name == 'search' and isinstance(inputs, dict):
    limit = inputs.get('limit', 5)
    if isinstance(limit, int) and limit > 10:
        return json.dumps(envelope(error='Safety rule blocked search: limit cannot exceed 10 records'))
    return json.dumps(call_domain_tool(name, inputs, repository), sort_keys=True)
```

ASSERTION EXAMPLES

```
assert result['ok'] and result['data']['count'] == 1
assert not invalid['ok'] and 'Invalid search input' in invalid['error']
assert not blocked['ok'] and blocked['error'].startswith('Safety rule blocked')
assert run.stop_reason == 'max_steps' and run.step_count == 3
```

OUTPUT

```
PASS search valid/invalid; detail valid/invalid; aggregate valid/invalid
PASS safety allowed/blocked; PASS agent max_steps
SUMMARY 9/9 tests passed
```

Safe execution entry point, assertion examples, and 9/9 summary.

```
< clear; .venv/bin/python scripts/test_hw05_tools.py
Vedant Vartak - DATA 260 Homework 5 offline tests
PASS test_search_valid
PASS test_search_invalid
PASS test_detail_valid
PASS test_detail_invalid
PASS test_aggregate_valid
PASS test_aggregate_invalid
PASS test_safety_rule_allowed
PASS test_safety_rule_blocked
PASS test_agent_max_steps
SUMMARY 9/9 tests passed
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

*Unaltered offline-test region cropped from the original terminal capture; name, all nine PASS results and summary remain visible.
The latest 9/9 rerun is recorded in RUN_LOG.txt.*

Part 5 - Bounded agent loop and Ollama scenarios

The agent loop code and corresponding four-scenario terminal output are shown together.

BOUNDED AGENT LOOP CODE AND OUTPUT

CODE

```
for step in range(1, max_steps + 1):
    action = active_model.next_action(messages)
    if 'final' in action: stop_reason = 'normal_completion'; break
    result = execute_tool(action['tool'], action['inputs'], repository)
    if not parsed_result['ok'] and error.startswith('Safety rule blocked'):
        stop_reason = 'safety_rule_block'; break
    else:
        stop_reason = 'max_steps'
```

OUTPUT

```
Scenario 1: steps=2, tool_calls=1, normal_completion
Scenario 2: steps=2, tool_calls=1, normal_completion
Scenario 3: steps=2, tool_calls=1, normal_completion
Scenario 4: steps=1, tool_calls=1, safety_rule_block
```

MAX_STEPS loop, execute_tool call, normal completion, and deterministic safety stop.

```
Vedant Vartak - DATA 260 Homework 5 agent scenarios
MODEL=qwen3:8b MAX_STEPS=6 SCENARIOS=4
Scenario 1 | steps 2 | tool_calls 1 | normal_completion
  Three shrimp recalls were found: 1. **Grocery Safety Manufacturer 10** (Produce) - Frozen Shrimp Lot 00001 (REC-353
9-00001), 6,535 units affected. 2. **Grocery Safety Manufacturer 09** (Dairy and Refrigerated) - Frozen Shrimp Lot 00
004 (REC-3539-00004), 13,996 units affected. 3.
Scenario 2 | steps 2 | tool_calls 1 | normal_completion
  {'product': 'Frozen Shrimp Lot 00001', 'manufacturer': 'Grocery Safety Manufacturer 10 (Safety Contact 10)'}
Scenario 3 | steps 2 | tool_calls 1 | normal_completion
  Recalls grouped by category (min_units=0): Packaged Foods had the highest total units (1,389,038) and 131 recalls.
Meat and Seafood followed with 130 recalls (1,327,145 units). Dairy and Refrigerated (127 recalls, 1,198,565 units) a
nd Produce (112 recalls, 1,098,442 units) comple
Scenario 4 | steps 1 | tool_calls 1 | safety_rule_block
  Safety rule blocked search: limit cannot exceed 10 records
PASS: 3 normal completions + 1 safety-rule block
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Three grounded completions plus one safety-rule block using qwen3:8b.

32. Agent reflection

Selected run: Scenario 4 - deterministic safety-rule block.

I selected the fourth Ollama run because it demonstrates that the harness, not the language model, owns the safety boundary. The user prompt explicitly asked the agent to retrieve recall records with a limit of 25 and told it not to reduce that limit. In step 1, local qwen3:8b produced a structured action for the search tool with query set to an empty string and limit set to 25. The harness did not send that action directly to the database. Instead, `run_agent` routed it through the single `execute_tool(name, inputs)` entry point used by every agent tool call.

Inside `execute_tool`, the domain-specific safety rule checked the requested search limit before normal schema validation or repository access. Because the limit exceeded the allowed agent maximum of 10, the function returned the shared JSON envelope with `ok=false`, `data=null`, and the error 'Safety rule blocked search: limit cannot exceed 10 records.' It raised no exception and issued no SQL query.

The loop parsed this clean result, logged the tool name, input, complete result, step number, and run identifier to `agent_runs.jsonl`, and recognized the safety-error prefix. It then stopped immediately with `stop_reason` equal to `safety_rule_block`. The run therefore used one step and one tool-call attempt, never approached the six-step ceiling, and never gave the model a chance to bypass the policy. This run shows why centralized enforcement is stronger than asking the model to follow a prompt: even when the model follows the user's unsafe request exactly, deterministic application code blocks it consistently.

Word count: 264

33. Tool contracts and AI-use disclosure

Tool	Expected JSON	Rejected JSON	Complete error location
search	<code>{"query": str, "limit": int}</code>	<code>{"query":"s","limit":5}</code>	Section 23: Domain MCP contract - search
detail	<code>{"recall_id": positive int}</code>	<code>{"recall_id":0}</code>	Section 25: Domain MCP contract - detail
aggregate	<code>{"group_by": enum, "min_units": int}</code>	<code>{"group_by":"submitter", "min_units":0}</code>	Section 27: Domain MCP contract - aggregate

Sections 23, 25, and 27 contain each full expected schema, valid input/output, rejected object, untruncated returned error, and explanation. All three return `{ok, data, error}`. Pydantic validation is caught at the boundary, and `execute_tool` applies the narrower agent-only limit of 10.

AI-use disclosure

1. I used an AI assistant extensively for implementation, tests, local tool operation, screenshot capture, report generation, and the final handout audit. I supplied my identity, assignment materials, review feedback, and earlier screenshots, and requested corrections. The assistant ran the automated checks and captured the final MealDB evidence; these are not claimed as independently performed manual work.
2. An unsuitable earlier report paired a random-meal Inspector screenshot with a different meal in its companion output. Earlier latency tables also mixed runs. Those outputs could not support the report's claims reliably.
3. Review feedback identified the mismatched meal IDs and latency values. During the correction, the assistant compared screenshots with machine-readable responses and checked the metrics against the raw CSV rather than relying on a generated summary alone.
4. I requested a corrected submission. The assistant replaced the MealDB captures and generated their readable panels from those same calls, retained the exact raw responses, and aligned the latency table to one recorded experiment. The delete-confirmation screen is an added safety/usability feature, not an explicit assignment requirement.

34. Submission file alignment

Required item	Repository location
Final report	reports/hw05/report.pdf
Run log	reports/hw05/RUN_LOG.txt
Metrics	reports/hw05/METRICS.md
AI disclosure	reports/hw05/AI_USE.md
Verification	reports/hw05/verification.json
Raw evidence	reports/hw05/raw/
Screenshots	reports/hw05/evidence/
Postman files	reports/hw05/postman/
Canvas upload copy	output/pdf/Vartak_HW5.pdf

Temporary UI/Postman demonstration records were deleted after proof was captured; the final authenticated API run also cleaned up its temporary fixtures. The final live database therefore matches the deterministic cumulative seed.

End of report